

# IMPERIAL

## HiPPO (2020)

Presnter: Xiqi Liang  
25/12/2025

# Slide 01 | Background

## Goal of sequence modeling

- Compress an **unbounded** history into a **fixed-dimensional** state.
  - Streaming data  $\Rightarrow$  long-range dependencies, but hardware is finite.
- 

## Why RNN naturally fits

$$h_t = \tau(h_{t-1}, x_t)$$

- $h_{t-1}$  summarizes the past;  $\tau$  merges the new input  $x_t$ .

## Limitation + mainstream variants

- **Vanishing/exploding** gradients  $\Rightarrow$  hard long-range memory.
- Variants: **LSTM/GRU** (gating), **orthogonal/unitary RNNs** (norm preservation).
- Still largely **heuristic**: what is stored is unclear; no optimality guarantees.

## So what?

- **Uncontrolled** under very long sequences or distribution shift (e.g., sampling-rate change).
- **No theory** that the state preserves the key information.

## HiPPO's shift

- Define “memory” as **function approximation** of the history.
- Derive a **closed-form** online update (ODE / recurrence).

## Slide 02 | HiPPO as a Whole

### What is HiPPO?

HiPPO is a **general memory layer** that replaces the heuristic "hidden state update" in RNNs. It maps a continuous stream to an optimal compressed representation.

### The I/O Interface

- **Input ( $f_t$ ):**

A time-varying stream (continuous function or discrete sequence). Typically the encoded features of raw input  $x_t$ .

- **Output / State ( $c(t)$ ):**

A fixed-dimensional coefficient vector  $c(t) \in \mathbb{R}^N$ .

\* This vector contains the **optimal projection** of the history up to time  $t$ .

### How to Plug it In

It integrates seamlessly into standard pipelines (like gated RNNs):

#### Step 1: Feature Encoding

$f_t = \text{Linear}(x_t)$  (Map raw input to feature space)

#### Step 2: HiPPO Memory Update

$c_t = A c_{t-1} + B f_t$  (Update memory state)

#### Step 3: Readout / Prediction

$y_t = \text{MLP}(c_t, f_t)$  (Generate output)

---

**Key Takeaway:** HiPPO only changes how the memory is updated (Step 2). The rest of the deep learning pipeline (encoding, decoding, backprop) remains exactly the same.

## Slide 03 | Why HiPPO Can Be a Better Memory

Why replace an LSTM/GRU-style memory update with HiPPO?

|                             | Traditional RNN                                                                                                                         | HiPPO                                                                                                                                                                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Interpretability</b>     | The hidden state $h_t$ is learned by gradient descent and behaves like a black box; it is hard to explain what a coordinate represents. | $c(t)$ is <b>provably optimal</b> (in an $L^2$ sense) projection coefficients.                                                                                                                                                                                  |
| <b>Closed-form dynamics</b> | Relies on nonlinear gating (sigmoid/tanh) to maintain memory, which can make optimization difficult (vanishing/exploding gradients).    | Memory update is governed by a <b>closed linear ODE</b> (state-space form), enabling efficient computation (often $O(N)$ with structure) and stable gradient propagation over very long sequences.                                                              |
| <b>Controllable bias</b>    | If we want “focus more on recent” vs “treat all past equally”, the model must learn this weighting implicitly.                          | We can explicitly choose a <b>measure</b> to control what history is emphasized: <ul style="list-style-type: none"><li>• <b>LegT</b>: sliding window (uniform)</li><li>• <b>LagT</b>: exponential decay</li><li>• <b>LegS</b>: full-history (uniform)</li></ul> |

# Slide 04 | HiPPO Framework: Pipeline

## High-Level Roadmap

$$f(t) \xrightarrow[\text{Measure}]{\text{Project}} c(t) \text{ (Coeffs)} \xrightarrow[\text{Reconstruct}]{} g^{(t)} \text{ (Approx)} \xrightarrow[\text{Differentiation}]{\text{Derive}} \textbf{Linear ODE}$$

**Step A | Continuous History:** Treat input sequence as a function  $f_{\leq t}$  in a Hilbert Space, focusing on history up to time  $t$ .

**Step B | Measure Family  $\mu^{(t)}$ :** Define "what history matters" via inner product  $\langle f, g \rangle_{\mu} = \int f(x)g(x)d\mu(x)$  (e.g., Uniform vs. Exp. decay).

**Step C | Polynomial Basis:** Project onto an N-dim subspace spanned by **Orthogonal Polynomials (OPs)**  $\{g_n^{(t)}\}$  for independent coefficients.

**Step D/E | Optimal Coefficients & Reconstruction:** Calculate state  $c(t) \in \mathbb{R}^N$  via projection  $c_n(t) := \langle f_{\leq t}, g_n^{(t)} \rangle_{\mu^{(t)}}$ . These coeffs define the optimal approximation  $g^{(t)}$ .

**Step F | Online Update (The Core Innovation):** Differentiation of  $c(t)$  yields a closed-form **Linear ODE**, avoiding integral re-computation:

$$\frac{d}{dt}c(t) = A(t)c(t) + B(t)f(t) \quad \implies \quad \text{Discrete: } c_{k+1} = A_k c_k + B_k f_k$$

# Slide 05 | HiPPO Framework: The Operator

## From Steps to Operators

We can abstract the pipeline (Steps A-F) into two core operators mapping history to state:

$$\mathbf{f}_{\leq t} \xrightarrow{\text{proj}_t} \mathbf{g}^{(t)} \xrightarrow{\text{coef}_t} \mathbf{c}(t)$$

### 1. The Projection Operator ( $\text{proj}_t$ )

Maps the history function  $\mathbf{f}_{\leq t}$  to the **polynomial subspace**  $\mathcal{G}$ . It finds the unique polynomial  $\mathbf{g}^{(t)}$  that minimizes the approximation error w.r.t. the measure  $\mu^{(t)}$ .

- **Mapping:**  $\text{proj}_t : \mathbf{f}_{\leq t} \mapsto \mathbf{g}^{(t)}$

### 2. The Coefficient Operator ( $\text{coef}_t$ )

Maps the polynomial  $\mathbf{g}^{(t)}$  to its **coordinate vector** in the basis. This produces the state vector  $\mathbf{c}(t) \in \mathbb{R}^N$  (which is effectively stored and updated via the ODE).

- **Mapping:**  $\text{coef}_t : \mathbf{g}^{(t)} \mapsto \mathbf{c}(t)$

---

**The HiPPO operator is the composition of these two steps:**

$$\text{hippo}(\mathbf{f})(t) = \text{coef}_t(\text{proj}_t(\mathbf{f}))$$

# Slide 06 | Measure Families: Defining Memory Bias

## The Degree of Freedom: Measure $\mu^{(t)}$

The measure determines which part of history the model emphasizes.  $d\mu^{(t)}(x) = \omega^{(t)}(x)dx$ .

| Measure                              | Density / Window                                           | Intuition                                                                                                    | Key Characteristics                                                                                         |
|--------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>LegT</b><br>(Translated Legendre) | <b>Sliding Window</b><br>Uniform on $[t - \theta, t]$      | <b>"Finite Memory"</b><br>Only cares about the last $\theta$ seconds. Older history is completely forgotten. | Good for fixed-range patterns, but cannot handle long-term dependencies beyond $\theta$ .                   |
| <b>LagT</b><br>(Translated Laguerre) | <b>Exponential Decay</b><br>$e^{-(t-x)}$ on $(-\infty, t]$ | <b>"Soft Forgetting"</b><br>Recent events matter most. History fades exponentially (like standard RNNs).     | Generalizes the gating mechanism of <b>LSTMs/GRUs</b> . Prioritizes recent information.                     |
| <b>LegS</b><br>(Scaled Legendre)     | <b>Full History</b><br>Uniform on $[0, t]$                 | <b>"Scale Adaptive"</b><br>The window stretches as time $t$ increases to cover everything from start to now. | <b>Best Performer.</b><br><b>Scale Invariant:</b> Robust to changes in sampling rates and timescale shifts. |

# Slide 07 | Why HiPPO-LegS? (1/2): The Timescale Problem

## The Challenge: Timescale Instability

Real-world sequences often exhibit mismatches between training and testing timescales. Standard models assume a fixed sampling rate or speed, leading to failure when these conditions change.

### Scenario A: Sampling Rate Shift

- **Setup:** Train a model on sensors sampled at **100Hz**.
- **Deployment:** Sensors are upgraded to **200Hz**.
- **Result:** A standard RNN (which learns dynamics based on a fixed step  $\Delta$ ) will perceive the input as "slowed down" or distorted, causing performance to collapse without retraining.

### Scenario B: Process Speed Variation

- **Setup:** Handwriting classification or speech recognition.
- **Variation:** The same character/word can be written **quickly** or **slowly**.
- **Result:** This creates sequences of vastly different lengths and dynamics for the same underlying semantic content.

---

### The Goal: Scale Equivariance (Robustness)

We need a memory mechanism that is invariant to input stretching/compressing:  
"If the input slows down, the memory update should automatically adapt, not break."

## Slide 08 | Why HiPPO-LegS? (2/2): The Scale-Adaptive Solution

### The Mechanism: A Stretching Window

LegS maps history onto a measure uniform over  $[0, t]$ . As time progresses, the window **automatically stretches** to cover the full history.

#### The Update Rule (Time-Varying SSM)

$$c_t = A_t c_{t-1} + B_t f_t$$

- **Parameter-Free (No  $\theta$ ):**

Unlike sliding windows, LegS has **no fixed window size**. It naturally adapts to any sequence length, eliminating the need to tune memory horizons.

- **Structured Time-Varying Dynamics ( $A_t, B_t$ ):**

The transition matrices are **NOT static** (like in RNNs) but evolve at every step  $t$ . They are **mathematically derived** to optimally update the projection coefficients without retraining.

---

**Result:** A memory model that is mathematically provable to be **Scale Invariant**.

# Slide 9 | Experimental Results: SOTA Performance

## Validation Strategy

We evaluated HiPPO-LegS across three critical dimensions: **Memory Capacity**, **Timescale Robustness**, and **Computational Efficiency**.

| Experiment                                                 | Task & Challenge                                                                                                                                                             | Key Result (Verification)                                                                                                                                                            |
|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Long-Range Dependency</b><br>(Permuted MNIST)           | <b>Task:</b> Classify images treated as flattened, shuffled pixel sequences (len=784).<br><b>Challenge:</b> Requires remembering early pixels over hundreds of steps.        | <b>SOTA Accuracy: 98.3%</b><br>Outperforms LSTMs, GRUs, and even <b>Transformers/Dilated CNNs</b> , proving superior memory retention without global attention mechanisms.           |
| <b>Timescale Robustness</b><br>(Character Trajectories)    | <b>Task:</b> Classify handwriting with <b>distribution shifts</b> (e.g., train on 100Hz, test on 200Hz).<br><b>Challenge:</b> Standard RNNs fail when sampling rates change. | <b>Scale Invariance Proved.</b><br>While ODE-RNNs dropped to ~40% accuracy, LegS maintained ~ <b>90%</b> , validating its theoretical time-scale robustness.                         |
| <b>Scalability &amp; Speed</b><br>(1M-Step Reconstruction) | <b>Task:</b> Reconstruct a white-noise signal of length <b>1,000,000</b> using only 256 units.<br><b>Challenge:</b> Extreme sequence length.                                 | <b>High Precision &amp; Speed.</b><br><b>MSE <math>\approx</math> 0.02</b> (vs LSTM 0.25).<br>Processed <b>470k elements/s</b> on a single CPU core ( <b>10x faster</b> than LSTMs). |

# Slide 10 | Generalization & Final Conclusion

## Beyond Long Memory: Generalization Capabilities

To verify HiPPO as a **general-purpose** sequence model (like LSTMs), we tested it on distinct domains: NLP and Chaotic Dynamics.

### 1. NLP: IMDB Sentiment

- **Task:** Standard text classification (variable length). Requires context understanding, not just rote memory.
- **Result: 87.8% Accuracy.** Matches/slightly beats LSTM (87.3%).
- **Takeaway:** HiPPO's specialized memory structure incurs **no "inductive bias tax"** on general tasks. It can replace LSTMs in standard pipelines.

### 2. Physics: Mackey-Glass

- **Task:** Predict chaotic time-series with complex non-linear dynamics.
- **Result:** Outperformed specialized LSTMs and LMUs. Reduced Normalized RMSE by **30%**.
- **Takeaway:** Captures complex continuous dynamics precisely, making it ideal for **physical modeling**.

---

## Summary of Contributions

### 1. Theoretical

Replaces heuristic gating with **Optimal Projection**.  
"Memory is a mathematical approximation, not a black box."

### 2. Methodological

Derives a fast **Linear Recurrence**.  
Update cost is merely  $O(N)$ , enabling massive efficiency gains over Transformers.

### 3. Innovation (LegS)

Achieves **Scale Invariance**.  
The first memory model robust to changing timescales and sampling rates without retraining.

## Slide 13 | HiPPo-LegS Derivation (1): The Scaled Measure

### From Static to Dynamic Windows

Previous measures (LegT, LagT) use a **static** view of the past (fixed length  $\theta$  or decay rate). **LegS (Scaled Legendre)** introduces a **dynamic measure** that stretches as time evolves.

#### The Scaled Legendre Measure Definition

The measure density is uniform over the interval  $[0, t]$ :

$$d\mu^{(t)}(x) = \frac{1}{t} \mathbb{I}_{[0,t]}(x) dx$$

#### Key Properties of this Definition:

- **Full History Coverage (Interval  $[0, t]$ ):**

Unlike sliding windows, LegS **never discards** information.

As  $t \rightarrow \infty$ , the support of the measure grows to cover the entire sequence from start ( $x = 0$ ) to now ( $x = t$ ).

- **Normalization Factor ( $\frac{1}{t}$ ):**

This factor ensures the total measure is always normalized:  $\int d\mu^{(t)} = 1$ .

Intuition: As history gets longer, the "importance" density of each specific moment decreases proportionally to maintain a constant total memory budget.

## Slide 14 | HiPPO-LegS Derivation (2): The Dynamic Basis

### Problem: Domain Mismatch

Standard Legendre polynomials  $P_n(x)$  are defined on the static interval  $[-1, 1]$ . However, LegS requires a basis orthogonal with respect to the **growing interval**  $[0, t]$ .

We define the basis  $p_n^{(t)}(x)$  by stretching  $P_n(x)$  to fit  $[0, t]$ :

$$p_n^{(t)}(x) := \sqrt{2n+1} P_n\left(\frac{2x}{t} - 1\right)$$

### Decomposing the Transformation:

- **Coordinate Transformation**  $\left(\frac{2x}{t} - 1\right)$ :

This term linearly maps the history time  $x \in [0, t]$  back to the reference domain  $[-1, 1]$ .

- **Normalization Factor**  $\sqrt{2n+1}$ :

This scaling ensures the basis remains **orthonormal** under the LegS measure  $\mu^{(t)}$ .

Without this, the norm of the basis functions would change as the interval stretches.

---

**Result:**  $c_n(t) = \langle f_{\leq t}, p_n^{(t)} \rangle_{\mu^{(t)}}$  represents the optimal projection of the full history onto this dynamically stretching basis.

## Slide 15 | HiPPO-LegS Derivation (3): Leibniz Differentiation

### Goal: Time-Derivative of Coefficients

Recall the projection definition for LegS (on interval  $[0, t]$  with measure  $\frac{1}{t}$ ):

$$c_n(t) = \int_0^t f(x) p_n^{(t)}(x) \frac{1}{t} dx$$

We need to compute  $\frac{d}{dt} c_n(t)$  to update the state online.

Applying the **Leibniz Integral Rule**: The time variable  $t$  appears in three places: the **integration limit**, the **measure scaling** ( $\frac{1}{t}$ ), and the **basis functions** ( $p_n^{(t)}$ ).

Therefore, the derivative splits into three distinct terms:

$$\begin{aligned} \dot{c}_n(t) &= \frac{d}{dt} \left( \frac{1}{t} \int_0^t f(x) p_n^{(t)}(x) dx \right) \\ &= \underbrace{f(t) p_n^{(t)}(t) \frac{1}{t}}_{(1) \text{ Boundary Term}} + \underbrace{\int_0^t f(x) p_n^{(t)}(x) \left( -\frac{1}{t^2} \right) dx}_{(2) \text{ Measure Scaling Term}} + \underbrace{\int_0^t f(x) \frac{\partial}{\partial t} p_n^{(t)}(x) \frac{1}{t} dx}_{(3) \text{ Basis Stretching Term}} \end{aligned}$$

\* We must now simplify these integrals to express them in terms of  $c(t)$  and  $f(t)$ .

## Slide 16 | HiPPO-LegS Derivation (4): Closing the Loop

### From Integrals to Matrix Dynamics:

We analyze the three terms derived from the Leibniz rule:

- **Term (1) & (2):**

The boundary term gives direct access to input  $f(t)$ . The measure term simplifies simply to  $-\frac{1}{t}c_n(t)$ .

- **Term (3) - The Key Algebraic Insight:**

The derivative of an orthogonal polynomial  $\frac{\partial}{\partial t}p_n^{(t)}$  is a polynomial of degree  $n$ . Thus, it can be expressed as a linear combination of the basis functions themselves:

$$\frac{\partial}{\partial t}p_n^{(t)}(x) = -\frac{1}{t} \sum_{k=0}^{n-1} M_{nk} p_k^{(t)}(x)$$

This allows us to substitute the integral back with coefficients  $c_k(t)$ .

---

### Final Result: The HiPPO-LegS ODE

Collecting all terms into vector form yields the final LTI system:

$$\frac{d}{dt}c(t) = -\frac{1}{t}Ac(t) + \frac{1}{t}Bf(t)$$

## Slide 17 | Dynamics (1): The Continuous ODE

### The Continuous-Time Evolution

Based on the derivation, the optimal coefficient vector  $c(t)$  evolves according to a **Linear ODE** with time-varying coefficients:

$$\frac{d}{dt}c(t) = -\frac{1}{t}Ac(t) + \frac{1}{t}Bf(t)$$

### The Structure of Matrices A and B:

Unlike standard RNNs, these matrices are **fixed constants** (derived from Legendre polynomials) and are NOT learned via gradient descent.

#### Input Vector B (Scaling)

$$B_n = (2n + 1)^{1/2}$$

(Injects input  $f(t)$  into all polynomial orders appropriately.)

#### State Matrix A (Interaction)

$$A_{nk} = \begin{cases} (2n + 1)^{1/2}(2k + 1)^{1/2} & \text{if } n > k \\ n + 1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

### Key Property: Lower Triangular Structure

Note that  $A_{nk} = 0$  for  $n < k$ . This **Lower Triangular** structure is the secret to efficiency—it ensures that higher-order coefficients do not feed back into lower-order ones, simplifying the discrete update.

## Slide 18 | Dynamics (2): The Discrete Recurrence

### The Discrete Update Rule

In reality, we discretize it (e.g., via Generalized Bilinear Transform) into a standard **Linear Recurrence**:

$$c_{k+1} = A_k c_k + B_k f_k$$

\* Here  $k$  is the discrete step count. Note that transition matrices  $A_k, B_k$  evolve with time.

### Why is this efficient?

- **Linear Complexity  $O(N)$ :**

Calculating  $c_{k+1}$  requires only a matrix-vector multiplication. Because  $A$  is structured (lower triangular / quasi-separable), this operation can be optimized to  $O(N)$ , independent of sequence length  $T$ . (Compare to Transformers which are  $O(T^2)$  for attention).

- **Constant Memory Footprint:**

We never need to store the full history  $f_{0:k}$ . The vector  $c_k$  (size  $N$ ) contains the **compressed sufficient statistics** of all past inputs.

- **Drop-in RNN Replacement:**

The form  $c_{k+1} = A_k c_k + B_k f_k$  is exactly a Linear RNN. It can be inserted into any standard deep learning pipeline as a feature extraction layer.

# Slide 19 | Properties of LegS

## Rigorous Mathematical Foundations

HiPPO-LegS is not just an empirical architecture; it comes with strong theoretical guarantees regarding robustness, speed, and learning stability.

| Property                                             | Mathematical Basis                                                                                                                                             | Practical Benefit                                                                                                                                         |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Timescale Robustness</b><br>(Scale Invariance) | <b>Scaling Factor <math>1/t</math>:</b><br>The memory operator commutes with time-scaling: $\text{hippo}(f(\alpha t)) = \text{hippo}(f)(\alpha t)$ .           | <b>Generalization:</b><br>The model handles <b>sampling rate shifts</b> (e.g., 100Hz $\rightarrow$ 200Hz) or speed variations without retraining.         |
| <b>2. Computational Efficiency</b>                   | <b>Structured Matrix A:</b><br>Since A is <b>Lower Triangular</b> and structured, the matrix-vector multiplication $A_k c_k$ simplifies.                       | <b>Blazing Fast Speed:</b><br>Reduces update cost from naive $O(N^2)$ to $O(N)$ . Allows using very large memory ( $N = 256+$ ) with negligible overhead. |
| <b>3. Gradient Flow &amp; Error Bounds</b>           | <b>Polynomial Decay:</b><br>Gradients decay polynomially $\Theta(1/t)$ , not exponentially like standard RNNs ( $e^{-t}$ ).<br>Error decays as $O(N^{-1/2})$ . | <b>Long-Term Memory:</b><br>Solves the <b>Vanishing Gradient Problem</b> . The model can optimize dependencies over millions of steps.                    |

**IMPERIAL**